1.	Organisatorisches	4
2.	Einleitung	22
3.	HTML	47
4.	CSS	71
5.	Frameworks	90
6.	HTTP	111
7.	Internet	137
8.	TypeScript	192
9.	**HTML+CSS+Typescript**	**238**
10.	Web Services	253
11.	REST-Services	273

Alles zusammen …

Sie wissen jetzt wie man

- Informationen in HTML strukturiert,
- mit CSS darstellt und
- TypeScript programmiert.

Wie aber passt alles zusammen?



Skripte in HTML

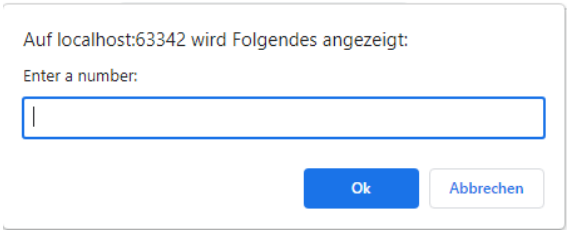
- Die einfachste (und unflexibelste) Möglichkeit ist, JavaScript direkt in die HTML-Datei einzufügen
- Beispiel

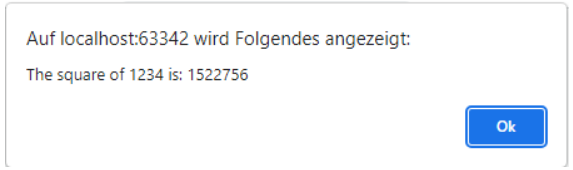
```
<!DOCTYPE html>
<html lang="de">
<head>
   <title>Square Calculator</title>
</head>
<body>
<h1>Square Calculator</h1>
<script>
      var userInput = prompt("Enter a number:");

      if (userInput !== null) {
         // User did not click 'Cancel' in the prompt

         var number = parseFloat(userInput);

         if (isNaN(number)) {
            alert("Invalid input. Please enter a valid number.");
         } else {
            var square = number * number;
            alert("The square of " + number + " is: " + square);
         }
      }
</script>
</body>
</html>
```





Diskutieren Sie mit Ihrem Nachbarn die Vor- und Nachteile!

Option 2: Script in Datei auslagern

```typescript
function calculateSquare() : number {
    let userInput: string | null = prompt("Enter a number:");

    if (userInput !== null) {
        // User did not click 'Cancel' in the prompt

        let number: number = parseFloat(userInput);

        if (userInput === null) {
            alert("Invalid input. Please enter a valid number.");
        } else {
            let square: number = number * number;
            alert("The square of " + number + " is: " + square);
        }
    }
}
```

TypeScript

```html
<!DOCTYPE html>
<html lang="de">
<head>
  <title>Square Calculator</title>
  <script src="../../../build/6_HtmlCssJavaScript/chap6_2_external_script.js"></script>
</head>
<body>
<h1>Square Calculator</h1>
  <button onclick="calculateSquare()">Calculate Square</button>
</body>
</html>
```

JavaScript!

Besser? Eigentlich wollen wir direkt die HTML-Seite ändern …

Zugriff auf Elemente im DOM („selektieren“)

- `document.getElementById("meine-id");` `// Rückgabetyp = HTMLElement`
- `document.getElementsByTagName("p");` `// HTMLCollection of HTMLElements`
- `document.getElementsByClassName("meine-klasse");`
- `document.querySelectorAll(".meine-klasse");`

Um einen konkreteren Typen zu bekommen, müssen wir mit dem **as** Operator den Typ selbst festlegen:

```
let fname: HTMLInputElement = document.getElementById("fname") as HTMLInputElement;
```

```
<!DOCTYPE html>
<html lang="de">
<head>
   <meta charset="UTF-8">
   <script src="../../../build/6_HtmlCssJavaScript/chap6_3_select.js"></script>
   <title>Title</title>
</head>
<body>
  <p id="Abschnitt1"> Hier ist der erste Abschnitt.</p>
  <p id="Abschnitt2">Und hier gleich der zweite.</p>
  <button onclick="abschnitt1()">Lies Abschnitt1</button>
  <button onclick="alleAbschnitte()">Lies alle Abschnitte</button>
</body>
</html>
```

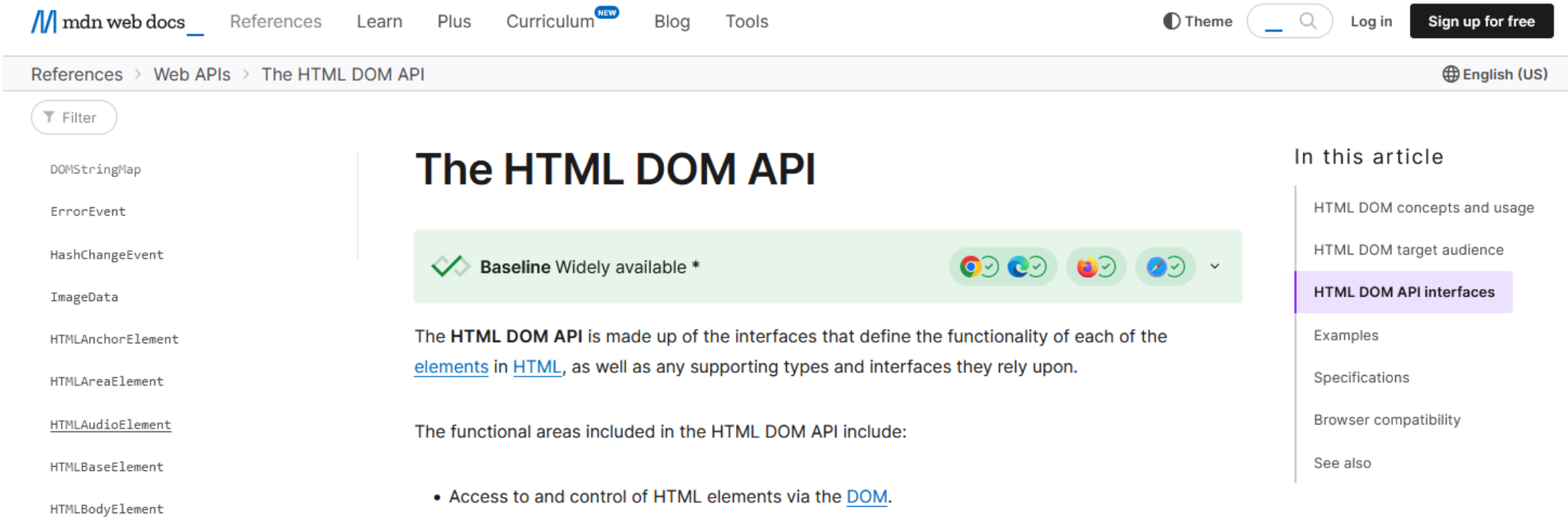
```
function abschnitt1() : void {
   const a1 : HTMLParagraphElement  = document.getElementById("Abschnitt1") as
HTMLParagraphElement ;

   alert("Abschnitt1 ist: " + a1.textContent);
}
function alleAbschnitte() : void {
   const paras  : HTMLCollectionOf<HTMLParagraphElement> = document.getElementsByTagName("p") ;

   for (let i=0; i<paras.length; i++){
      alert((paras[i] as HTMLParagraphElement).textContent);
   }
}
```

HTML DOM API

Einen sehr guten Überblick über das HTML DOM API finden Sie unter:
https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API



Elemente ändern

- Um ein Element zu ändern, wählt man es aus (`getElement…`), speichert es in einer Variablen und ändert die Eigenschaften des Elementes.
- Beispiel:

```
<!DOCTYPE html>
<html lang="de">
<head>
    <meta charset="UTF-8">
  <script
src="../../../build/6_HtmlCssJavaScript/chap6_4_changeElements.js">
</script>
    <title>Title</title>
</head>
<body>
<div id="element">
  Hier ist (schon wieder) Text.
</div>
<button onclick="changeElement()">Ändern</button>
</body>
</html>
```

```
function changeElement() {
    const element: HTMLDivElement = document.getElementById("element") as
                    HTMLDivElement;            // kompiliert nur wenn strictNullChecks
 = false
    element.textContent = "Oh, nein!"
    element.style.fontWeight = "bold";
}
```

Elemente hinzufügen

- Um ein Element zu hinzufügen, wählt man das Parent-Element aus, in dem es hinzugefügt werden soll, und hängt das neue Element mit append an das Parent-Element an.
- Beispiel:

```
<div id="element">
    Some text.
</div>
<button onclick="appendElement()">Hinzufügen</button>
```

```
function appendElement() {
    const parent: HTMLDivElement = document.getElementById("element") as HTMLDivElement;
        // kompiliert nur wenn strictNullChecks = false
    const newChild = document.createElement("p");
    newChild.textContent = "Even more text";
    parent.append(newChild);

}
```

Elemente löschen

- Um ein Element zu löschen, selektiert man es und ruft die `remove-Methode` auf.
- Beispiel:

```
div id="element">
  <p>Dies ist ein Paragraph</p>
</div>
<button
onclick="removeElement()">Löschen</but
ton>
```

```
function removeElement() {
    const elemnt: HTMLDivElement =
          document.getElementById("element") as HTMLDivElement;
          // kompiliert nur wenn strictNullChecks = false
    elemnt.remove();
}
```

Styles ändern

Um schnell das Aussehen eines Elementes zu ändern, kann man den Style mittels inline-CSS direkt ändern (nicht empfohlen!) oder eine CSS-Klasse hinzufügen/entfernen.

1. Selektieren des Elementes
   ```
   const element = document.getElementById("meinElement");
   ```

2. Style setzen:
   ```
   if (element) { element.style.backgroundColor = "red"; … }
   ```

Oder:

3. CSS-Klasse(n) hinzufügen:
   ```
   if (element) { element.classList.add("class1", "class2", …); }
   ```

   Entfernen mit remove("class1", "class2", …);
   oder: element.classList="";

Events

- Eine dynamische Veränderung des DOMs wird häufig durch ein vorheriges Event ausgelöst. Ein Event wird meistens (nicht immer!) durch eine Aktion des Benutzers ausgelöst:

  Abschicken eines Formulars, Klick auf einen Button, Eingabe in Input-Feldern, Scrollen, …
- TypeScript erlaubt es, eigene Aktionen an Events zu knüpfen, indem man einen Event-Handlers registriert.
- Dazu benutzt man `addEventListener`, definiert den Event-Typ (hier: `"click"`) und hinterlegt einen Code-Block mit Aktionen, welche nach Eintritt des Events ausgeführt werden sollen.
- Diese Vorgehensweise ist besser als <button onclick=…>, weil Sie so Darstellung (html) und Logik (TS) trennen können.

```
const searchButton = document.getElementById("search-button");
searchButton.addEventListener("click", () => {
  const searchInput: HTMLInputElement = document.getElementById("search") as HTMLInputElement;
  let searchText = searchInput.value;
  console.log(searchText);
});
```

Achtung!

- Wenn Sie das Skript der Vorseite in den Header einbinden, wird es ausgeführt, bevor das DOM aufgebaut ist. `search-button` wird dann noch den Wert `null` haben!
- Lösung: Das Event DOMContentLoaded wird ausgelöst, wenn das DOM vollständig aufgebaut ist, so dass wir einen zweiten `EventListener` hinzufügen können:

```
document.addEventListener("DOMContentLoaded", function() {
        const searchButton = document.getElementById("search-button");
        searchButton.addEventListener("click", () => { … }) });
```

All together now

- Wir wollen ein einfaches Formular erstellen mit einem Input-Feld und einem Button.
- Wenn der Benutzer einen Text eingegeben hat, bewirkt das Klicken auf den Button, dass der eingegebene Text an eine Tabelle unterhalb des Formulars angehängt wird.
- Überlegen Sie, wieviele Elemente Sie benötigen und was der EventListener tun soll.

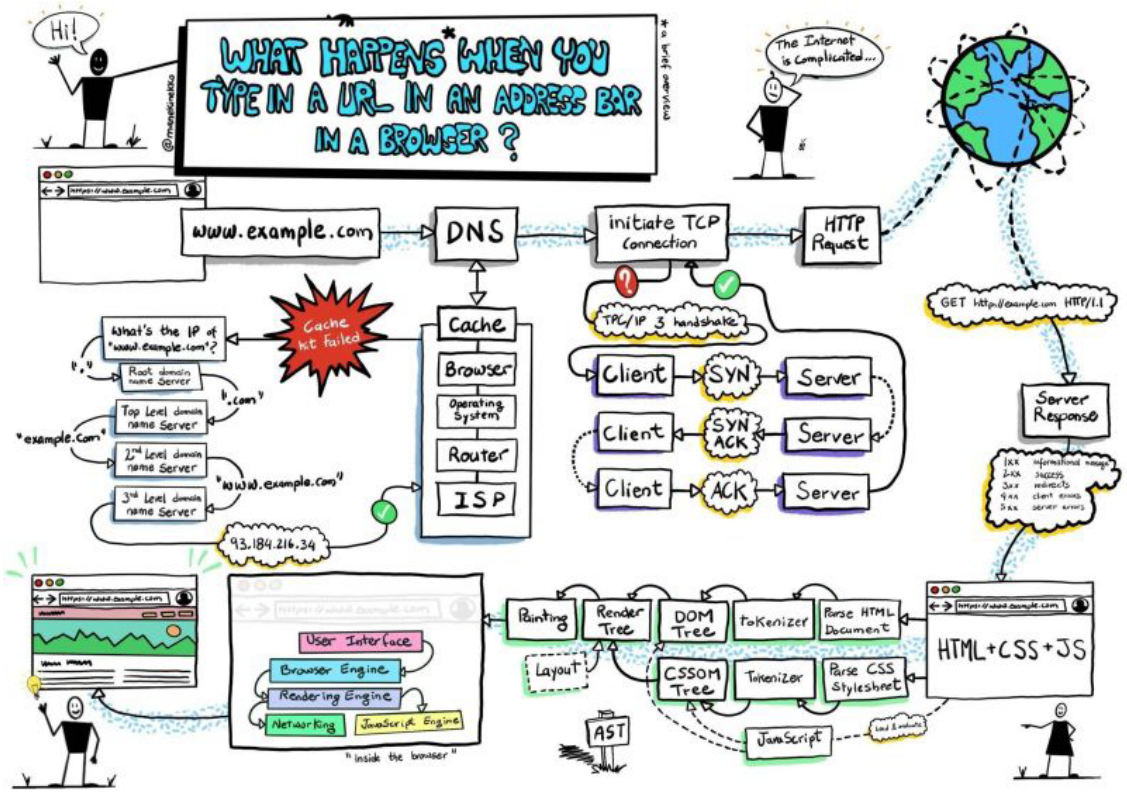
```
<form id="addForm">
  <input type="text" id="inputText"
placeholder="Enter text">
  <button type="button"
id="addButton">Add</button>
</form>
<table id="dataTable">
  <tr>
    <th>Tabelle</th>
  </tr>
</table>
</body>
```

```
document.addEventListener("DOMContentLoaded", function() {
    const addButton = document.getElementById("addButton") as HTMLButtonElement;
    const inputText = document.getElementById("inputText") as HTMLInputElement;
    const dataTable = document.getElementById("dataTable") as HTMLTableElement;

    addButton.addEventListener("click", function() {
        const text = inputText.value.trim();        // Text aus Eingabefeld lesen
        if (text !== "") {
            const newRow = dataTable.insertRow(dataTable.rows.length);
            const cell = newRow.insertCell(0);      // Zelle an Tabelle anfügen
            cell.textContent = text;
            inputText.value = ""; // Eingabefeld leeren
        }
    });
});
```

Wo sind wir?

- Alles in dem u.a. Bild haben wir behandelt … aber nur für statischen Inhalt (HTML + CSS), der in Dateien steht.
- Wir wollen aber auch dynamischen Inhalt erzeugen, der zum Beispiel aus Datenbanken kommt.



Hinter der URL muss also auf dem Server ein Programm stehen, das auf einen Request antwortet.

Hier geht es im Oktober weiter!

Ziele und Inhalte (aus Modulhandbuch)

Lerninhalt

Die Studierenden kennen die grundlegenden Konzepte client- und serverseitiger Web-Programmierung, einschließlich der Darstellung und Übertragung von Information im Internet. ☑

Inhalte:

- Grundlagen der Client-Server-Programmierung und ‚des Internets' ☑
- Informationsdarstellung/-Übertragung: Markup Languages, Protokolle, Sessions ☑
- Client-seitige Programmierung mit TypeScript ☑

Qualifikations- und Lernziele

Fachkompetenzen

- Die Studierenden können die Grundlagen der **Client-Server-Programmierung** beschreiben.
- Sie können verschiedene **Arten der Darstellung von Information** in Client-Server-Systemen beschreiben und voneinander abgrenzen.

Methodenkompetenzen (fachlich & überfachlich)

- Die Studierenden sind in der Lage, ein **einfaches webbasiertes System** auf Client-Seite zu entwerfen, umzusetzen und Entwurfsentscheidungen zu begründen.
- Sie können **Informationen** dem Verwendungszweck angemessen **darstellen**.

Sozialkompetenzen

- Die Studierenden können die vorgestellten Grundlagen in **Teamarbeit** umsetzen und ihre Ergebnisse in Übungsgruppen **präsentieren**.

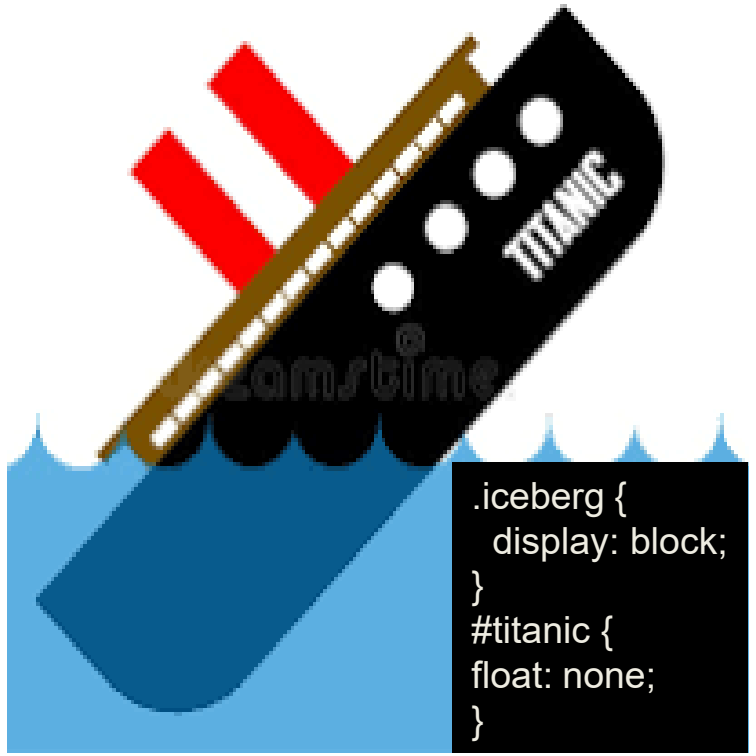
Selbstkompetenzen

- Die Studierenden können die erlernten Konzepte strukturiert auf neue Problemstellungen **anwenden**.

Kurze Zusammenfassung

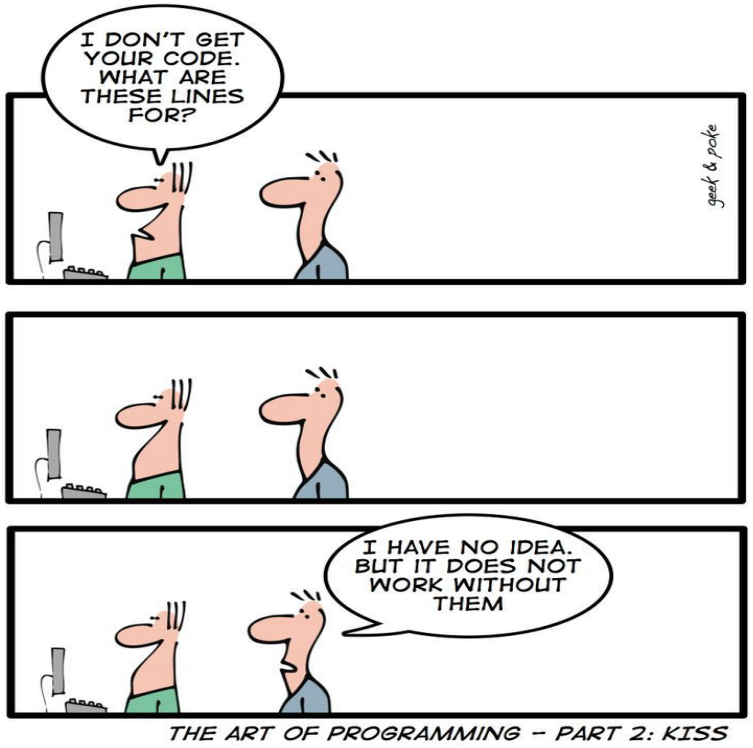
HTML und CSS





Kurze Zusammenfassung

JavaScript / TypeScript



Klausur

- **16.7.2025, 10:30 Uhr in Hessenhalle 1 und 3.**
- Bitte finden Sie sich **30 Minuten vorher** vor Halle 1 ein und bringen Sie Ihren **Studierendenausweis** mit.
- Wir führen eine Einlasskontrolle durch – **nur angemeldete Personen dürfen mitschreiben**.
- Als Hilfsmittel ist nur ein handgeschriebener DIN A4-Zettel zulässig (Kopien oder Ausdrucke sind nicht erlaubt).
- Die Klausur dauert 90 Minuten.
- Die Korrektur wird ca. 2-3 Wochen dauern, danach erhalten Sie eine E-Mail und sehen Ihr Ergebnis im eCampus. Eine Klausureinsicht findet zeitnah statt.

